Sound for Silence

SYNOPSIS

OF MAJOR PROJECT

BACHELOR OF TECHNOLOGY

Computer Science Engineering

SUBMITTED BY

Abhishek Verma Roll No. 22EAOCS005

Shekhar Chaurasiya Roll No. 22EAOCS047

Sakshi Puria Roll No. 22EAOCS042

ARYA INSTITUE OF ENGINEERING, TECHNOLOGY AND MANAGAMENT, JAIPUR

MAY 2026

TABLE OF CONTENTS

1. Introduction 1
2. Objectives 2
    2.1. Literature Review 3
3. Feasibility Study 4
4. Methodology / Planning of Work 5
5. Facilities Required for Proposed Work 8
6. Expected Outcomes 9

1. Introduction

Sound for Silence is a dual-interface digital healthcare platform designed specifically for children undergoing cognitive, auditory, and speech therapy. The platform targets children who have received cochlear implants and need structured, monitored rehabilitation over an extended period. The system consists of two separate but connected interfaces: an Android mobile application for patients and their parents, and a web-based dashboard for therapists and clinicians.

The **Android App**, built using Kotlin and Jetpack Compose, acts as a Patient and Parent Portal. It presents therapy content in the form of gamified learning stages and sequential videos. Each video must be completed before the next one is unlocked, ensuring that the child follows a structured learning path. After each video, a short quiz tests comprehension. Parents can log in to view their child's progress, including day streaks, completion percentages, and stage-wise performance.

The **Web Dashboard**, developed using HTML, CSS, and JavaScript, serves as a Therapist Portal. Therapists can log in to view their assigned patients, check implant dates, and see upcoming assessment schedules. They can record clinical scores such as the CAP score (Categories of Auditory Performance, 0–7) and the SIR score (Speech Intelligibility Rating, 0–5) for each session. Longitudinal progress charts built using Chart.js allow therapists to visualize improvement over 3-month, 6-month, and 12-month periods.

The entire backend is powered by Firebase. Firebase Authentication handles secure login for both parents and therapists using email and password, with role-based access control. Firebase Firestore stores all application data including user profiles, patient records, stage content, video metadata, quiz results, and clinical assessments. Firebase Hosting serves the web dashboard. No separate server or Node.js backend is required.

Rationale: Children with cochlear implants require consistent, guided practice over months and years. Currently, therapy progress is often tracked manually using paper records, which makes longitudinal monitoring difficult and error-prone. Sound for Silence addresses this gap by providing a structured learning environment for the child, transparent progress visibility for the parent, and data-driven clinical tools for the therapist.

2. Objectives

- To develop an Android application that delivers sequential, gamified therapy content to children with cochlear implants and tracks their learning progress automatically.
- To build a web-based therapist dashboard that allows clinicians to manage patient records, record clinical scores (CAP and SIR), and visualize long-term progress using interactive charts.
- To implement a unified Firebase backend that provides secure, role-based authentication and real-time data synchronization between the Android app and the web dashboard without requiring a dedicated server.
- To design a parent-facing portal that provides clear visibility into their child's daily activity, quiz performance, and overall rehabilitation progress.

2.1. Literature Review

S.No.	Source / Title / Tool	Key Idea	Limitations / Gap	Relevance to Project
1	Cochlear Implant Rehab Apps (e.g., AB CLIX, Hearoes)	Mobile apps for auditory training games for CI children.	No therapist portal; no clinical score recording; no longitudinal charts.	Motivates the need for a connected dual-interface system with clinical monitoring.
2	Firebase Firestore Documentation (Google)	NoSQL real-time database with offline support and role-based security rules.	Requires careful data modelling; complex queries need composite indexes.	Used as core database for users, patients, stages, videos, and assessments.
3	Jetpack Compose Documentation (Android Developers)	Declarative UI framework for Android enabling composable, reactive interfaces.	Steeper learning curve than XML layouts; limited legacy library support.	Forms the UI foundation of the Android patient/parent portal.
4	Chart.js Documentation	JavaScript library for interactive data visualisation charts on the web.	Limited built-in analytics; needs external processing for complex insights.	Used to render CAP/SIR progress charts (3-month, 6-month, 12-month) on the therapist dashboard.
5	Research Paper — "Mobile Health Apps for Paediatric Audiology"	Reviews mHealth tools in paediatric hearing rehabilitation.	Most apps lack real-time therapist oversight or standardised scoring.	Supports CAP/SIR tracking design and the need for therapist-facing tools.

3. Feasibility Study

Technical Feasibility: The proposed system is technically feasible using widely available and well-documented technologies. The Android application is built using Kotlin and Jetpack Compose, both officially supported by Google with extensive documentation and community support. The web dashboard uses standard HTML, CSS, and JavaScript with the Chart.js library, which runs in any modern browser without additional setup. The entire backend relies on Firebase, a managed cloud platform that eliminates the need for a dedicated server, handles authentication, stores data in real time, and hosts the web dashboard. All chosen tools are lightweight, scalable, and have been successfully used in production healthcare and education applications. The technical complexity is well within the scope of a final-year engineering project.

Operational Feasibility: The system is designed to be simple and intuitive for all three types of users — children, parents, and therapists. The child-facing portion of the Android app uses a game-like interface with large buttons, bright visuals, and short videos to keep young users engaged. Parents can track progress from a clean dashboard without requiring any technical knowledge. Therapists can enter CAP and SIR scores through a straightforward form and view charts with a single click. The automated sync between the app and the dashboard removes the need for manual data entry or paper records, saving time and reducing errors. The system is operationally practical for clinics, rehabilitation centres, and home use.

Economic Feasibility: The project is economically feasible because all core tools and platforms are free or low-cost. Kotlin, Jetpack Compose, Android Studio, HTML, CSS, JavaScript, and Chart.js are open-source and free to use. Firebase offers a generous free tier (Spark Plan) that covers authentication, Firestore reads/writes, and hosting — sufficient for development, testing, and small-scale deployment. No dedicated server or paid hosting is required. The only potential cost is a Google Play Store developer account (one-time fee of approximately USD 25) for publishing the Android app. Overall, the development and deployment costs are minimal, making the system economically viable for a student project and affordable for real-world clinical adoption.

4. Methodology / Planning of Work

The development of Sound for Silence follows an **Agile Software Development Life Cycle (SDLC)** approach. Agile methodology allows the project to be built in short, focused iterations, with regular testing and refinement at each stage. This is especially useful for a dual-interface system where the Android app and web dashboard must remain in sync throughout development.

Phase 1 — Requirement Analysis: In this phase, all functional and non-functional requirements are identified. Functional requirements include sequential video unlock logic, post-video quizzes, parent progress dashboard, therapist patient management, CAP/SIR score entry, and Chart.js progress visualisation. Non-functional requirements include real-time data sync, secure role-based login, offline support for the Android app, and a responsive web interface. Use cases are defined for each user role — child, parent, and therapist.

Phase 2 — Firebase Setup and Data Modelling: Firebase project is created and configured for both Android (Firebase Android SDK) and web (Firebase JS SDK v9 modular). Firestore collections are designed to store users, patients, stages, videos, quiz results, and clinical assessments. Firebase Authentication is configured with email/ password login and role-based access control using custom claims. Security rules are written to ensure each user can only access data relevant to their role.

Phase 3 — Android App Development: The Android application is developed using Kotlin and Jetpack Compose. Key features implemented include: stage listing with unlock logic, video playback using ExoPlayer/Media3, post-video quiz screens, and a parent dashboard displaying streaks, completion percentage, and progress. Firebase Android SDK is integrated for authentication, Firestore data access, and offline caching.

Phase 4 — Web Dashboard Development: The therapist portal is built using HTML, CSS, and JavaScript with the Firebase JS SDK v9 modular API. Pages include a patient list view, individual patient detail view with implant and assessment dates, a score entry form for CAP and SIR values, and a Chart.js-powered progress chart page. The dashboard is hosted on Firebase Hosting.

Phase 5 — Integration and Testing: Both interfaces are tested end-to-end to ensure data entered on one platform reflects correctly on the other. Unit tests are conducted

for quiz scoring logic and score validation. User acceptance testing is performed with sample data simulating real patient scenarios.

Phase 6 — Deployment: The web dashboard is deployed to Firebase Hosting. The Android APK is built in release mode and prepared for distribution. Project documentation is finalised.

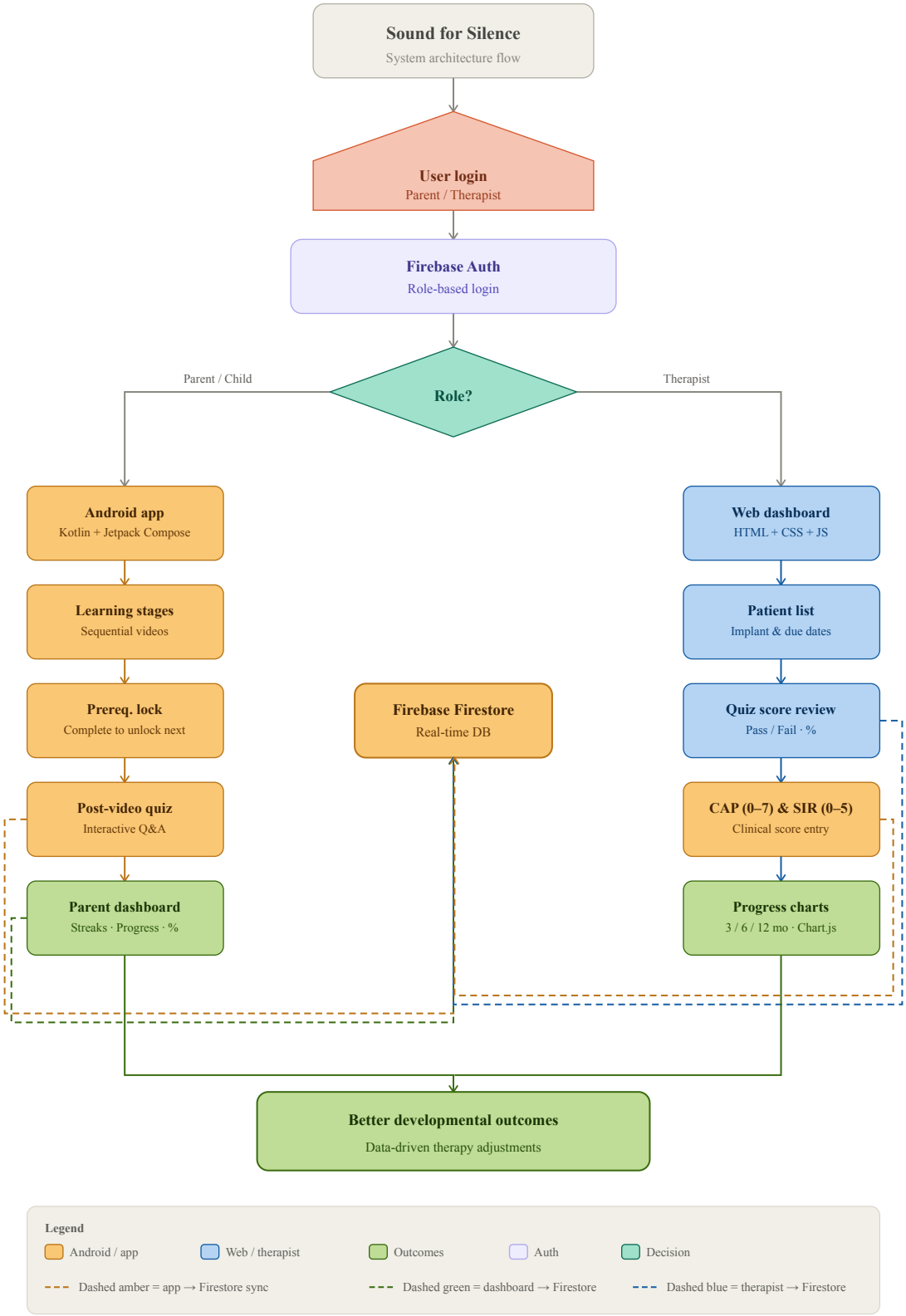

Figure 1: Agile Development Phases — Sound for Silence

5. Facilities Required for Proposed Work

The development of Sound for Silence requires standard hardware and software resources that are readily available. On the hardware side, a computer with an Intel Core i5 or i7 processor, 8 to 16 GB of RAM, and at least 512 GB of storage is recommended to run Android Studio, the Android emulator, and a browser-based development environment simultaneously without performance issues. A physical Android device is needed for real-device testing of the app, as the emulator does not fully replicate audio playback behaviour. A stable internet connection is required for Firebase synchronisation, downloading dependencies, and accessing Firebase Console during development. On the software side, Android Studio is used as the primary IDE for Android development, with Kotlin 1.9+ and Jetpack Compose for building the app UI and logic. ExoPlayer (via Jetpack Media3) handles video playback within the app. The Firebase Android SDK connects the app to Firestore and Authentication. For the web dashboard, Visual Studio Code is used as the editor, with plain HTML, CSS, and JavaScript as the frontend stack. The Firebase JS SDK v9 modular API handles all backend communication, and Chart.js is used for progress visualisation. Firebase Hosting serves the dashboard. Git is used for version control throughout the project. Google Chrome and its DevTools are used for testing and debugging the web interface.

6. Expected Outcomes

Upon successful completion, the Sound for Silence platform will deliver a fully functional Android application that provides cochlear implant children with a structured, gamified therapy experience through sequential videos and post-video quizzes, with automatic prerequisite unlock logic ensuring they follow the correct learning path. Parents will have access to a real-time dashboard showing their child's daily streaks, stage completion percentages, and overall progress without any manual input from the therapist. The web-based therapist portal will allow clinicians to manage their assigned patients, record CAP scores (0–7) and SIR scores (0–5) per session, and view interactive Chart.js progress charts over 3-month, 6-month, and 12-month periods, enabling evidence-based clinical decisions. All data will synchronise in real time between the Android app and the web dashboard via Firebase Firestore, eliminating paper records and reducing administrative workload. The system will demonstrate practical skills in Android development with Kotlin and Jetpack Compose, cloud-based backend design using Firebase, web frontend development, and data visualisation — reflecting a complete, end-to-end software engineering workflow applicable to real-world healthcare technology.